Deep Creativity

Logan Foster, John Berberian, Sami Adam (Group 5)

Agenda

- Problem Definition
- Deep Creativity Metric Updates
- Deep Creativity VAE
- Diffusion
- Results
- Conclusion

Problem Definition

Generative models tend to memorize training data

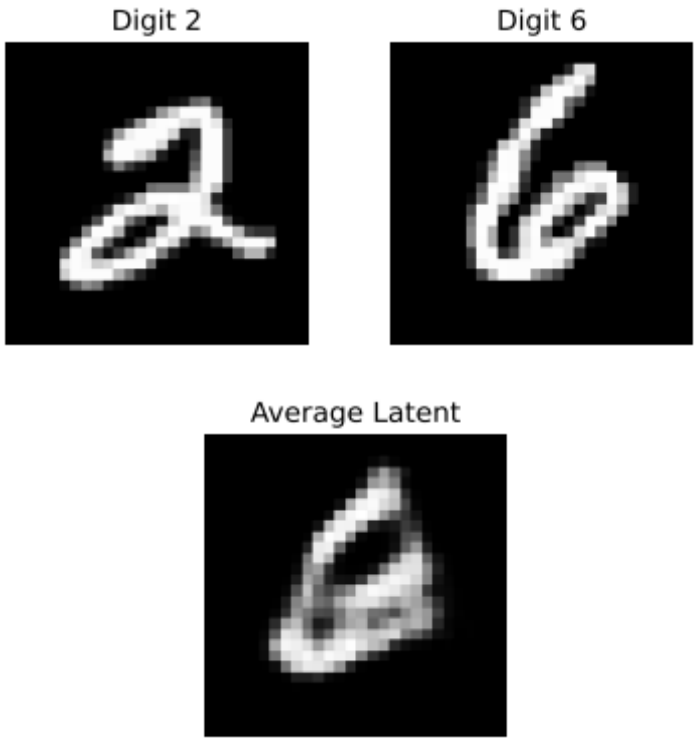
- Creates privacy concerns
- Conflicts with the broader goal of generating novel data

Generation of Intermediate Digits

- Between 2 and 6
- With some quality-creativity metric
- Compared to latent interpolation of an autoencoder

Challenges

- Generating novel outputs without incentivizing noise
- Defining quality and creativity
- Defining superiority over the baseline



DC metric captures value, novelty, and surprise

DC Metric

$$DC(x) = V(x)^{a_1} \cdot N(x)^{a_2} \cdot S(x,z)^{a_3},$$

- Weighted geometric mean
- DC bounded to be in [0,1)

Value

$$V(x) = \sigma(f_{\text{value}}(x)) \in [0,1]$$

- Probability from digit classifier

Novelty

$$N(x) = H(\tilde{p}_2, \tilde{p}_6)^a \cdot (p_2 + p_6)^r \in [0,1]$$

$$H(\tilde{p}_2, \tilde{p}_6) = -\frac{\tilde{p}_2 \log \tilde{p}_2 + \tilde{p}_6 \log \tilde{p}_6}{\log 2} \quad \tilde{p}_c = \frac{p_c}{p_2 + p_6}$$

- P̃: weighted logit
- Mix of confusion (entropy) and relevance

Surprise

$$S(x,z) = 1 - \exp(-\lambda \cdot D_{\text{KL}}(q_\phi(z \mid x) \parallel p(z)))$$

$$S(x,z) \in [0,1)$$

- High KL divergence leads to large surprise

Transition to connection to VAE

DC Metric

$$\mathcal{L} = -E_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] + \beta \cdot D_{\mathrm{KL}} - (a_1 \log V + a_2 \log N + a_3 \log S)$$

- DC integrates into ELBO well, as surprise is also KL divergence

Models

- **Base VAE:**
  - [784, 512, 256, 32] to [32, 256, 512, 784]
  - Bernoulli decoder
- **Creative VAE:** Copy of Base VAE

Latent Space Hotspot Search

$$\{\mu_k, \sigma_k, \pi_k\}^* = \arg\max_{\{\mu_k, \sigma_k, \pi_k\}} \sum_{k=1}^{K} \pi_k \cdot E_{\varepsilon \sim \mathcal{N}(0,I)}[\log DC(p_\theta(\mu_k + \sigma_k \odot \varepsilon))] + \lambda H(\pi)$$

$$\text{subject to: } \|\mu_k\|_2 \le \sqrt{d}, \quad \sigma_k \in [\sigma_{\min}, \sigma_{\max}]^d, \quad \textstyle\sum_k \pi_k = 1, \quad \forall k$$

- Without L2 constraint, gradient ascent pushed means to large-norm regions where decoder is untrained
  - After each step, mean is projected into ball
- Entropy term maximized at uniform weights, lambda trades off exploitation vs over-concentration

VAE and Latent Hotspot Search Training

VAE + Creative VAE Training Schedules

- **Base VAE:**
  - trained for 40 epochs
- **Creative VAE:**
  - Trained for 40 more epochs
  - Alternating ELBO and log DC loss (each had individual optimizers)

Experiments

- **A:** Score Generated (sampling latent space)
  - Sampling vs image recon. creativity
- **B:** ELBO/DC Epoch Ratio
  - Control push and pull of ELBO and DC
- **C:** DC KL Penalty
  - Control push and pull of ELBO and DC

Latent Hotspot Search

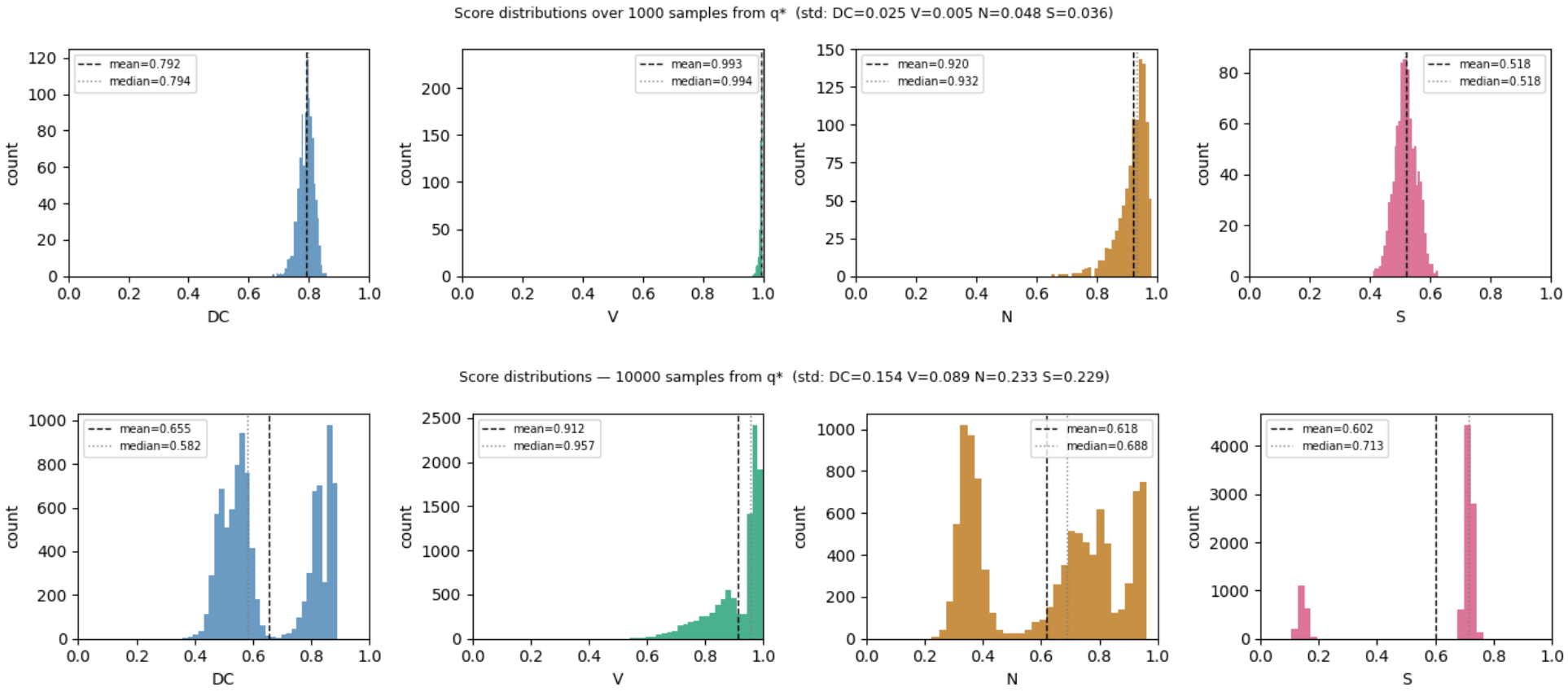
- VAE and classifiers are fully frozen
- Instead of point estimate, local gaussian mixtures fit
- Reparameterization trick applied per component
- All k components are initialized at latent interpolation with small random perturbations added to K – 1 components
- Per component L2 and log sigma projections

$$z_k = \mu_k + \sigma_k \odot \varepsilon_k,$$

$$\varepsilon_k \sim \mathcal{N}(0, I), \qquad k = 1, \ldots, K$$

Increasing K in latent search allows for more variation

Creative VAE Latent Search (K=1 vs K=5)



Diffusion

Base Diffusion Model

- We trained a Classifier-Free DDPM diffusion model based on Ho et al. (2022) using a UNET architecture.
- Probability of unconditioned denoising was 0.1.
- Prasad et al. (2023) showed that UNET's connections can help improve denoising.

Two-Six Denoising Process

- Lui et al. (2023) proposed composing scores across different classes to generate clean, out-of-domain samples.
- We implement this by computing the delta between our unguided noise and predicted noise given a two or six class.

Paper

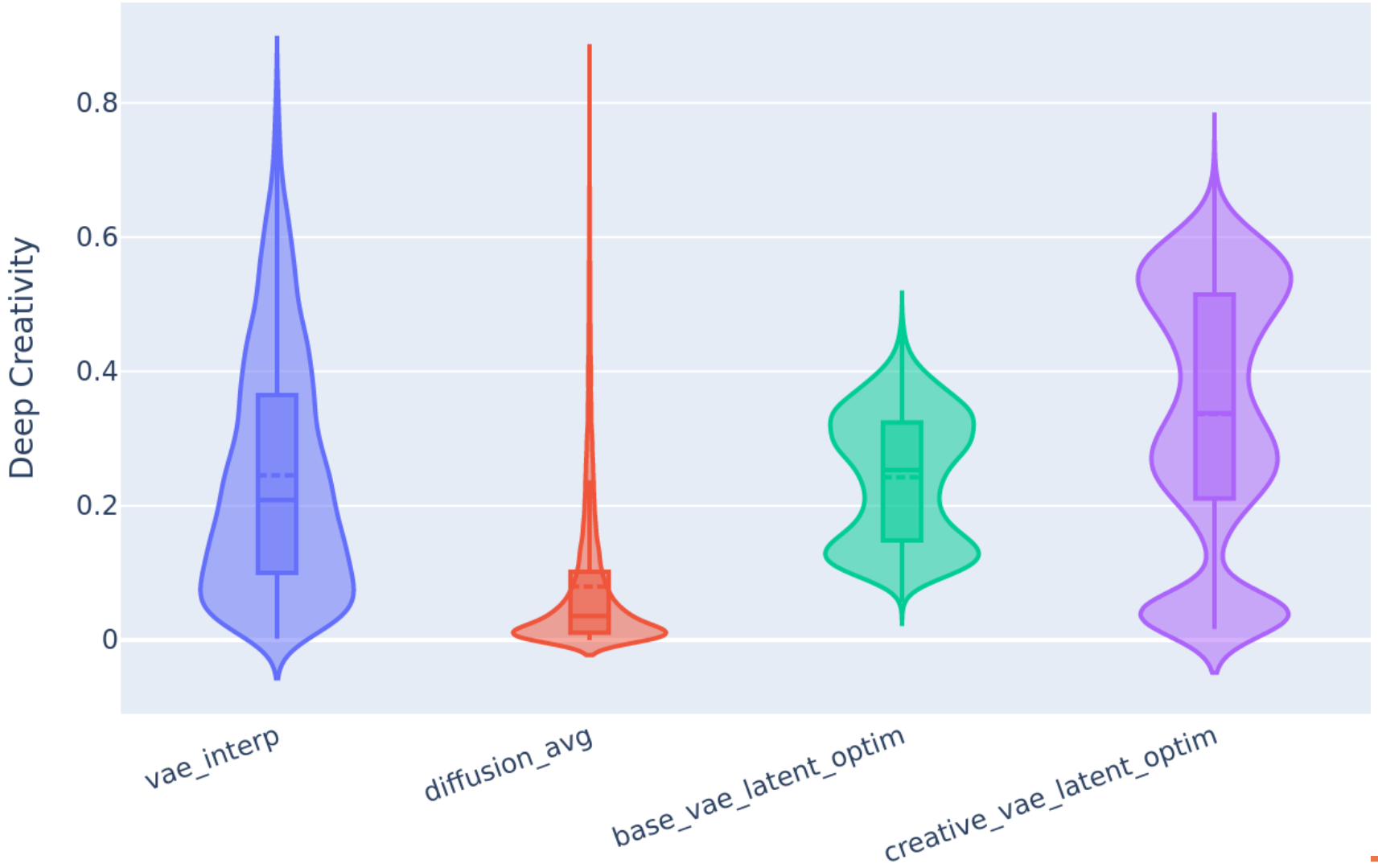
$$\hat{\epsilon}(\boldsymbol{x}_t, t) = \epsilon_\theta(\boldsymbol{x}_t, t) + \sum_{i=1}^{n} w_i \left(\epsilon_\theta(\boldsymbol{x}_t, t|\boldsymbol{c}_i) - \epsilon_\theta(\boldsymbol{x}_t, t)\right)$$

Us

$$\hat{\epsilon} = \epsilon_\emptyset + \frac{s}{2}\left[(\epsilon_2 - \epsilon_\emptyset) + (\epsilon_6 - \epsilon_\emptyset)\right]$$

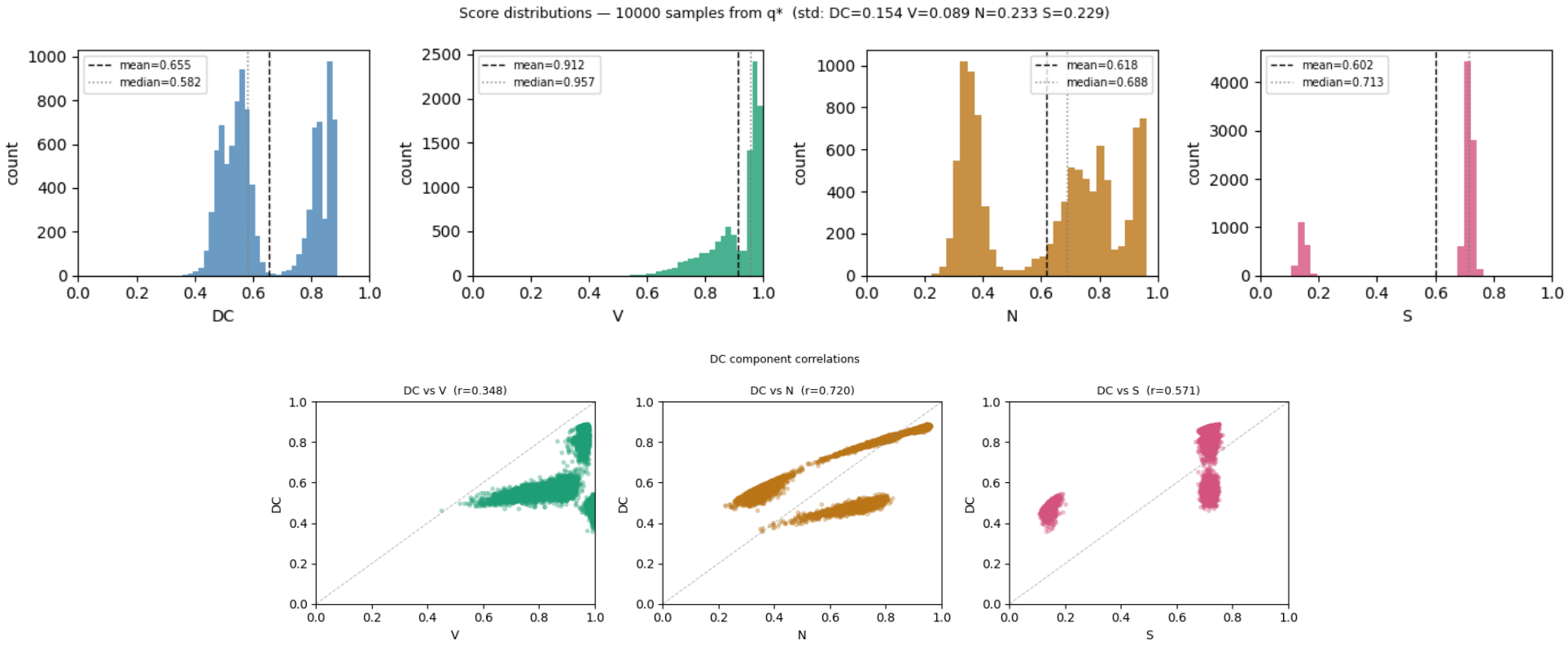
```
def create_eps(model, z, t_tensor, c1_cls, c2_cls, null_cls, mode, guidance_scale):
    z3 = torch.cat([z, z, z])
    t3 = torch.cat([t_tensor, t_tensor, t_tensor])
    c3 = torch.cat([c1_cls, c2_cls, null_cls])

    eps_all = model(z3, t3, c3)
    eps_c1, eps_c2, eps_uncond = eps_all.chunk(3, dim=0)

    d1 = eps_c1 - eps_uncond
    d2 = eps_c2 - eps_uncond

    return eps_uncond + guidance_scale * 0.5 * (d1 + d2)
```

Results (Scores)



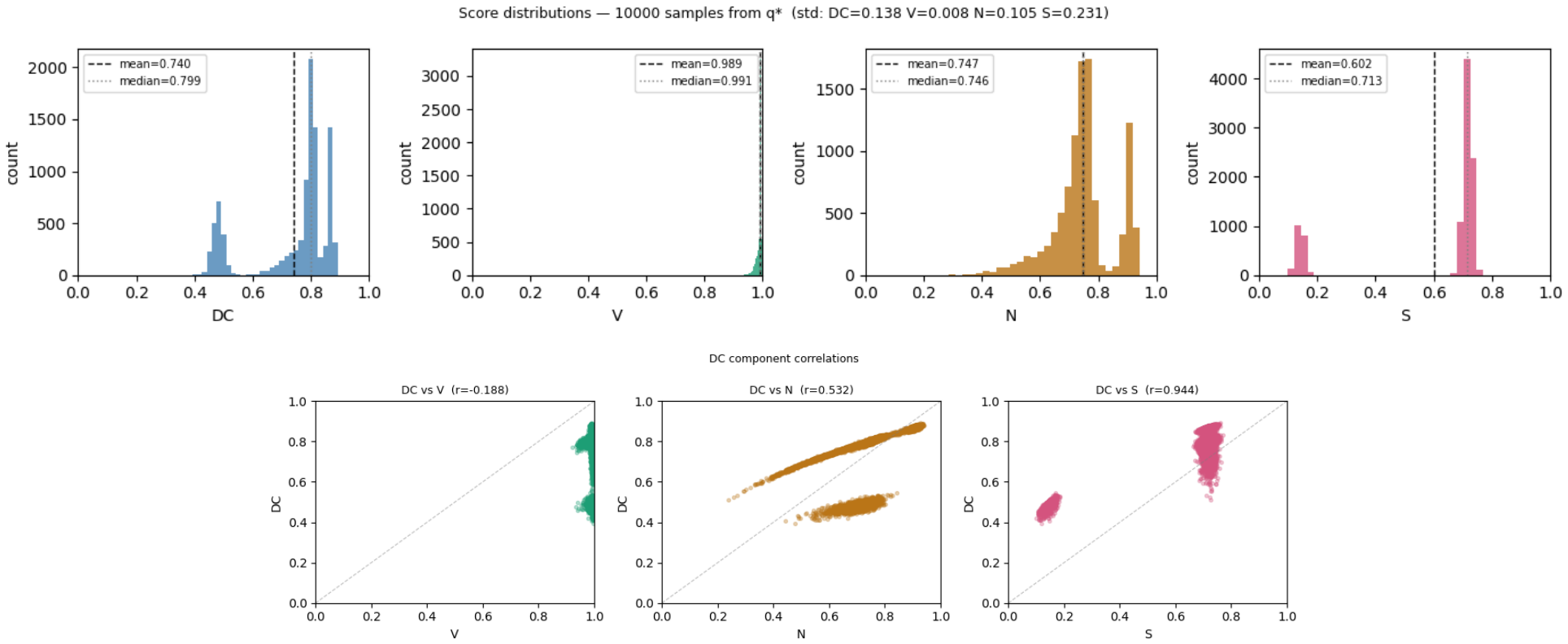
Latent Hotspot Search Results (i)

Baseline VAE Latent Search

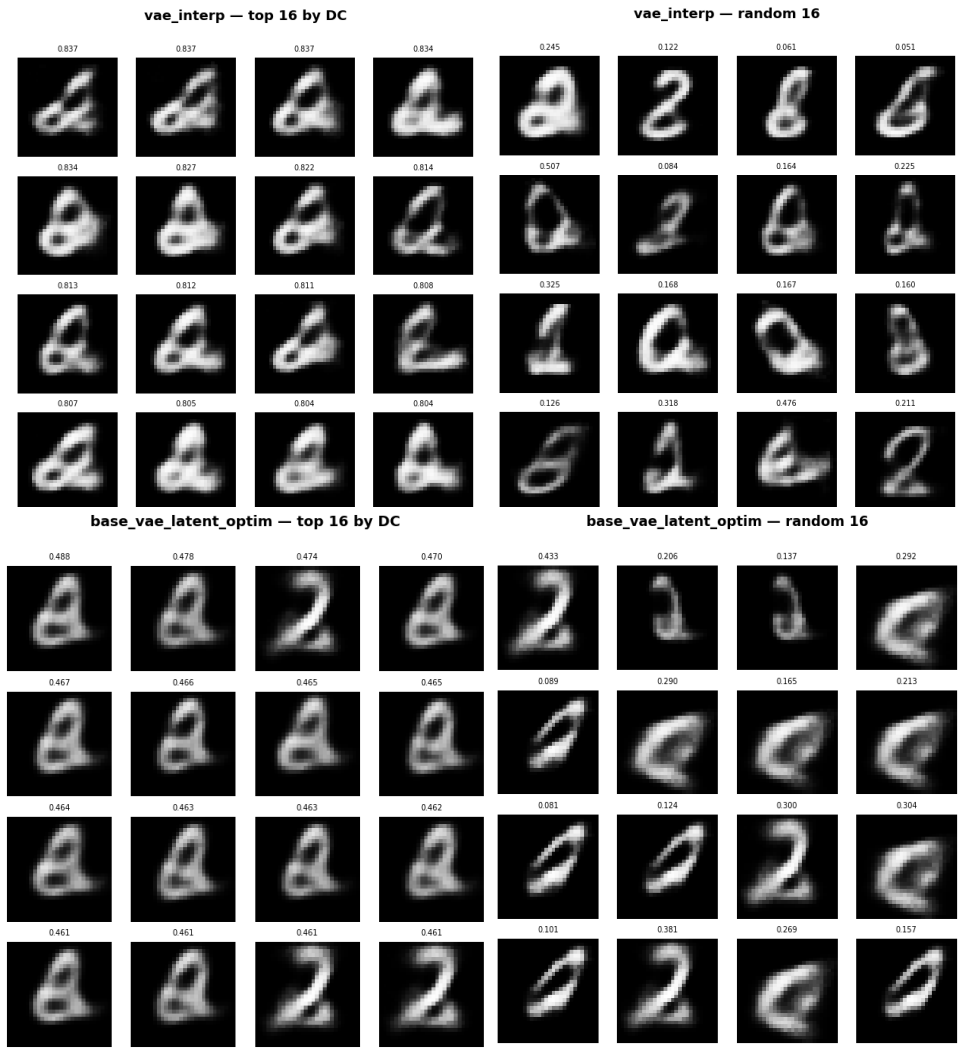


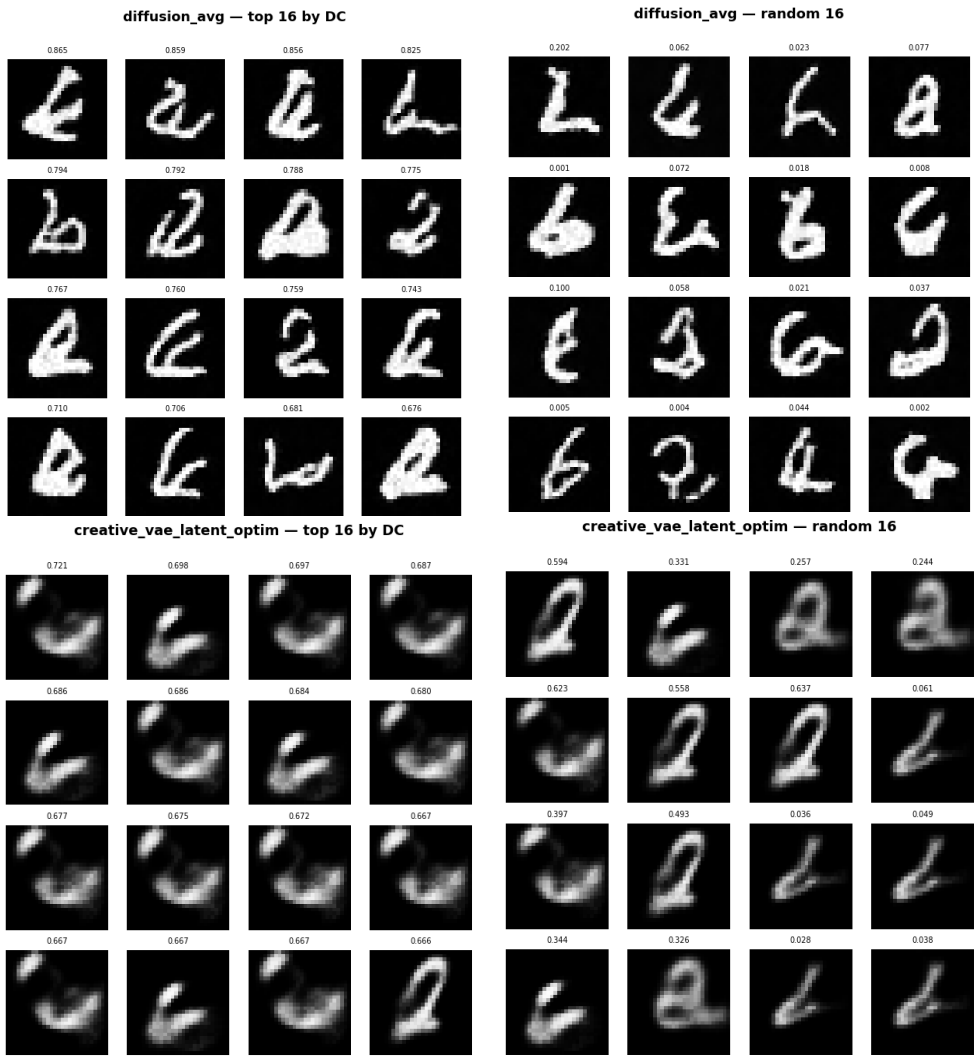
Latent Hotspot Search Results (ii)

Creative VAE Latent Search



Results (Visualizations)





Take Aways

Metric Performs Great

- We were happy to see our metric successful filter for creative and meaningful samples across multiple different distribution strategies.
- We were surprised to see the diffusion model perform relatively poorly overall.

Penalize Similar Outputs

- We noticed that our models sometimes collapsed into mostly producing unhelpful models.
- We believe we could fix this by penalizing batches with very similar outputs.